第2章: TypeScript入門

> アプリの「動き（ロジック）」はすべて **TypeScript** という言語で書きます。この章では、後の章で出てくるコードを読み書きできるようになるための、TypeScriptの基礎を完全初心者向けに解説します。**初めて出てくる文法はすべて1つずつ説明**します。

---

1. TypeScript とは何か

1.1 JavaScript と TypeScript の関係

世の中のWebやアプリの多くは **JavaScript（ジャバスクリプト）** という言語で動いています。**TypeScript（タイプスクリプト）** は、その JavaScript に「**型（かた、type）**」という仕組みを足した、上位互換の言語です。

- **JavaScript** … 古くからある、型のない言語。手軽だがミスに気づきにくい。
- **TypeScript** … JavaScriptに型を足した言語。Microsoft社が開発。書いた瞬間にミスを発見できる。

> 「**上位互換**」とは？ TypeScriptはJavaScriptの全機能を含みつつ、さらに型を足したもの、という意味です。だからJavaScriptが書ければTypeScriptも書けますし、最終的にTypeScriptは「型を取り除いてJavaScriptに変換（コンパイル）」されてから実行されます。

1.2 「型」とは何か — みかん箱のたとえ

型（type） とは「この箱（変数）には、この種類のデータしか入れてはいけない」というルールです。

> **身近な例え:** 「みかん専用」と書かれた箱に、りんごを入れようとすると「それはみかんじゃないよ！」と止めてくれる。これが型の働きです。プログラムでも、文字を入れるべき場所に数値を入れようとすると、**実行する前に**エラーで教えてくれます。
>
> **▼ このコードがやること（先に日本語で）:** 「型なし（JavaScript）」と「型あり（TypeScript）」で、同じ足し算を書いたときの違いを見比べます。型がないと、文字列 `"1"` と数値 `2` を足しても気づかずに変な結果（`"12"`）になってしまいますが、型を付けると**実行する前に**「それは数値じゃない」とエラーで教えてくれます。ここで押さえるのは「型は実行前にミスを見つけてくれる仕組み」という1点だけでOKです（各行の詳しい意味はコード内のコメントにあります）。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
// ================================================================================
// 「型あり」と「型なし」で何が違うかの比較
// ================================================================================
// で始まる行は「コメント」。プログラムとしては実行されず、説明用に書ける。
// 注:

// ---------- 型なし（JavaScript）の場合 ----------
// function = 関数を作るキーワード / add = 関数名 / (a, b) = 受け取る材料2つ
function add(a, b) {
  // return = 結果を返す / a + b = 足し算（だが…）
  return a + b;
}
// "1"は文字列、2は数値。JSは文字列連結して "12" を返してしまう（バグ！）
add("1", 2);

// ---------- 型あり（TypeScript）の場合 ----------
// a: number → aは数値型のみ / : number（最後）→ 戻り値も数値
function addTs(a: number, b: number): number {
  // a も b も数値なので、必ず正しい足し算になる
  return a + b;
}
// ← "1"は文字列なので、ここで「型エラー」が表示され、実行前にミスに気づける
addTs("1", 2);
// VS Codeでは addTs("1", 2) の "1" の下に赤い波線が引かれ、次のメッセージが出る：
// Argument of type 'string' is not assignable to parameter of type 'number'.
// （'string'型の引数は'number'型のパラメータに代入できません）
```

> 「コンパイル」とは？ 人間が書いたTypeScriptを、コンピュータが実行できるJavaScriptに**変換する処理**のこと。この変換のときに型チェックが行われ、間違いがあればエラーになります。「実行前のチェック」だと考えてください。

---

2. 変数 — 値に名前を付けて入れておく箱

2.1 `const` と `let`

変数（へんすう、variable） とは「値に名前を付けて保存しておく箱」です。TypeScriptでは主に2つのキーワードで変数を作ります。

▼ **このコードがやること（先に日本語で）**：`const`（コンスト）という書き方で「あとから中身を変えない箱」を作り、そこに文字列を入れます。`=`（イコール）は「右の値を左の箱に入れる」という代入の記号で、数学の"等しい"ではない点に注意です。まずは「`const 名前 = 値;` で値に名前を付けて保存できる」という形だけ覚えればOKです（各記号の意味はコード内のコメントで説明しています）。

コピー

```
const title = "リーダブルコード";
// const  : 「後から中身を変えない箱」を作るキーワード（constant＝定数 の略）
// title  : 変数名（箱の名前）。自分で好きに付けられる。半角英数字で、小文字始まりが慣習
// =      : 「右の値を左の箱に入れる」という代入の記号（イコールだが"等しい"ではない）
// "リーダブルコード" : 入れる値。ダブルクォートで囲むと「文字列」になる
// ;      : 文の終わりを示すセミコロン
```

▼ **このコードがやること（先に日本語で）**：今度は `let`（レット）という書き方で「あとから中身を変えられる箱」を作り、中身を `0` から `1` に書き換えます。`const` との違いは「変更できるかどうか」だけで、`const` は変更しようとするとエラーになります。「変えたいときだけ `let`、それ以外は `const`」と覚えておけば十分です。

コピー

```
// let : 「後から中身を変えられる箱」を作るキーワード / 0 は数値
let count = 0;
// 中身を 0 から 1 に変更できる（letだから可能）
count = 1;
// もし上が const count = 0; だったら、count = 1; の行でエラーになる（constは変更不可のため）
```

`const` と `let` の使い分け: 迷ったら**まず `const` を使う**のが鉄則です。「変更しない」とコードで宣言しておくことで、うっかり書き換えるミスを防げます。本当に後で変える必要がある場合だけ `let` にします。（昔の `var` というキーワードもありますが、現在は使いません。）

---

3. 基本の型

TypeScriptでよく使う基本的な型を紹介します。変数名の後ろに `: 型名` と書くと、その変数の型を明示できます。

▼ **このコードがやること（先に日本語で）**：よく使う3つの基本の型（文字列・数値・真偽値）を、それぞれ変数に入れて見せています。変数名の後ろに付いている `: string` や `: number` の部分が「型注釈」で、「この箱には何の種類のデータが入るか」を宣言しています。`boolean` は `true`（はい）か `false`（いいえ）の2択しか入らない型、と覚えてください（各行の意味はコード内のコメント参照）。

コピー

```
// string（ストリング）= 文字列の型
const bookTitle: string = "達人プログラマー";
// number（ナンバー）= 数値の型（整数・小数どちらも）
const publishYear: number = 1999;
// boolean（ブーリアン）= 真偽値の型。true（真）か false（偽）の2択
const isRead: boolean = true;
// : string や : number の部分が「型注釈（type annotation）」。「この箱の中身の種類」を宣言している
```

▼ コードを1つずつ分解して解説

解説1: 文字列の箱を作る（`: string`）

コピー

```
// string（ストリング）= 文字列の型
const bookTitle: string = "達人プログラマー";
```

- `const bookTitle` は「あとから中身を変えない箱に `bookTitle` という名前を付ける」部分です。
- `: string` が**型注釈**です。変数名の直後に `: 型名` と書くことで「この箱には文字列しか入れない」と宣言しています。
- `= "達人プログラマー"` で、その箱に実際の文字列を入れています。ダブルクォート `"..."` で囲んだものが文字列です。
- もし後で `bookTitle = 123;` のように数値を入れようとすると、型注釈のおかげで**実行前にエラー**になります。

> **用語:** 型注釈（type annotation）＝変数名の後ろに `: 型名` を書いて「中身の種類」を明示する書き方。

解説2: 数値の箱を作る（`: number`）

コピー

```
// number（ナンバー）= 数値の型（整数・小数どちらも）
const publishYear: number = 1999;
```

- `: number` は「この箱には数値しか入れない」という型注釈です。
- `number` 型は**整数も小数も同じ**ように扱えます。`1999` でも `3.14` でも `number` です。
- 数値はクォートで囲みません。`"1999"` と書くと文字列になってしまい、`number` 型の箱には入れられません。

> **用語:** number（ナンバー）＝数値を表す型。整数・小数の区別はなく、どちらも number。

解説3: はい/いいえの箱を作る（`: boolean`）

コピー

```
// boolean（ブーリアン）= 真偽値の型。true（真）か false（偽）の2択
const isRead: boolean = true;
```

- `: boolean` は「この箱には `true` か `false` のどちらかしか入れない」という型注釈です。
- `boolean` 型に入る値は `true`（**真＝はい**）と `false`（**偽＝いいえ**）**の2つだけ**です。
- 変数名を `isRead`（読んだか？）のように `is` **で始める**と「はい/いいえで答えられる値」だと一目で分かるため、よく使われる命名です。

> **用語:** boolean（ブーリアン）＝ `true` / `false` の2択しか持たない型。条件判定の結果などに使う。

型	意味	例
`string`	文字列（文字の並び）	`"こんにちは"`, `"鈴木"`
`number`	数値（整数・小数）	`42`, `3.14`, `2024`
`boolean`	真偽値（はい/いいえ）	`true`, `false`
`null`	「値が無い」ことを表す特別な値	`null`
`undefined`	「まだ値が設定されていない」状態	`undefined`

> **型注釈は省略できる:** 実は `const title = "本"` と書くだけで、TypeScriptは「右が文字列だから title は string 型だ」と**自動で推測**してくれます（これを「型推論」と呼びます）。なので毎回 `: string` と書く必要はありません。本書では分かりやすさのため、要所で型注釈を明示します。

---

4. 配列とオブジェクト — 複数のデータをまとめる

4.1 配列（はいれつ、array）— 値の並び

配列 は「同じ種類の値を順番に並べたリスト」です。`[ ]`（角カッコ）で囲みます。

> **▼ このコードがやること（先に日本語で）:** 3つの文字列を `[ ]`（角カッコ）で並べて「配列（リスト）」を作り、その中から1つ取り出して表示します。`string[]` は「文字列がいくつか並んだリスト」という型です。**最大のポイントは「数え方が0から始まる」こと**—— `titles[0]` が1番目、`titles[1]` が2番目になります（取り出し方の詳細はコード内コメントにあります）。

コピー

```
const titles: string[] = ["本A", "本B", "本C"];
// string[] ： 「文字列(string)の配列([])」という型。「文字列がいくつか並んだリスト」の意味
// [ ... ] ： 配列を作る記号。中身はカンマ , で区切る
// 並んだ各値を「要素（element）」と呼ぶ

// titles[0] → 配列の「0番目」の要素を取り出す。結果: "本A"
console.log(titles[0]);
// 注意: プログラミングの数え方は0から始まる！ 1番目=[0]、2番目=[1]、3番目=[2]
// console.log(...) : ( )の中身をターミナルやデバッグ画面に表示する命令。動作確認によく使う
```

▼ コードを1つずつ分解して解説

解説1: 文字列の配列を作る（`string[]`）

コピー

```
const titles: string[] = ["本A", "本B", "本C"];
```

- `: string[]` が配列の型注釈です。`string`（文字列）の後ろに `[]` を付けると「**文字列がいくつも並んだリスト**」という意味になります。
- `[ ... ]`（角カッコ）が配列を作る記号です。中身の各値は**カンマ `,` で区切り**ます。
- ここでは `"本A"` `"本B"` `"本C"` の3つの文字列が、この順番で並んでいます。並んだ1つ1つの値を「要素（element）」と呼びます。

> **用語:** 配列（array）＝同じ種類の値を順番に並べたリスト。`型[]` でその要素の型を表す。

解説2: 配列から1つ取り出す（番号は0から）

コピー

```
// titles[0] → 配列の「0番目」の要素を取り出す。結果: "本A"
console.log(titles[0]);
```

- `titles[0]` の `[0]` は「**配列の何番目を取り出すか**」を指定する番号（インデックス）です。
- プログラミングの数え方は **0から始まる**のが最大の注意点です。`[0]` が1番目、`[1]` が2番目、`[2]` が3番目になります。
- そのため `titles[0]` は先頭の `"本A"` を取り出します。`titles[2]` なら `"本C"` です。
- `console.log(...)` は、カッコの中身を画面（ターミナルやデバッグ画面）に表示する命令で、動作確認によく使います。

> **用語:** インデックス（index）＝配列の要素の位置を表す番号。0から始まる。

4.2 オブジェクト（object）—「キーと値」の組

オブジェクト は「名前（キー）と値の組」をまとめたものです。`{ }`（中カッコ）で囲みます。1冊の本の情報をまとめるのに最適です。

> **▼ このコードがやること（先に日本語で）**：1冊の本の情報（タイトル・著者・出版年・読了したか）を、`{ }`（中カッコ）で1つにまとめた「オブジェクト」を作り、その中の値を1つ取り出して表示します。`title: "..."` のように「キー（項目名）：値」の形で項目を並べるのが特徴です。値を取り出すときは `book.title` のように `.`（**ドット**）**で項目名をつなぐ**、という1点を押さえてください（詳細はコード内コメント参照）。

コピー

```
const book = {
  // title というキー（項目名）に "リーダブルコード" という値
  title: "リーダブルコード",
  // author というキーに著者名。各組はカンマ , で区切る
  author: "Dustin Boswell",
  // year というキーに数値
  year: 2012,
  // isRead というキーに真偽値
  isRead: true,
};
// { } : オブジェクトを作る記号 / キー: 値 の形で項目を並べる

// book.title → オブジェクトの title の値を取り出す。結果: "リーダブルコード"
console.log(book.title);
// . （ドット）: 「オブジェクトの中の、その名前の項目」にアクセスする記号
```

▼ コードを1つずつ分解して解説

解説1:「キー: 値」の組をまとめてオブジェクトを作る

コピー

```
const book = {
  // title というキー（項目名）に "リーダブルコード" という値
  title: "リーダブルコード",
  // author というキーに著者名。各組はカンマ , で区切る
  author: "Dustin Boswell",
  // year というキーに数値
  year: 2012,
  // isRead というキーに真偽値
  isRead: true,
};
```

- `{ ... }`（中カッコ）がオブジェクトを作る記号です。配列の `[ ]` とは別物なので混同しないよう注意します。
- 中身は **`キー: 値` の形**で書きます。`title` `author` `year` `isRead` が「キー（項目名）」、その右側が「値」です。
- 各組（キーと値のセット）は**カンマ `,` で区切り**ます。
- 1つのオブジェクトの中に、**文字列・数値・真偽値など異なる種類の値を混ぜて**持てます。配列が「同じ種類の並び」なのに対し、オブジェクトは「1つのモノの複数の属性」をまとめるのに向いています。

> **用語:** オブジェクト（object）＝「キー（項目名）と値」の組を `{ }` でまとめたデータ。1冊の本など「1つのモノ」を表すのに使う。

解説2: ドット `.` で中の項目を取り出す

コピー

```
// book.title → オブジェクトの title の値を取り出す。結果: "リーダブルコード"
console.log(book.title);
```

- `book.title` の `.`（ドット）が「**オブジェクトの中の、その名前の項目にアクセスする**」記号です。
- `book.title` は「`book` というオブジェクトの中の `title` という項目の値」を指し、結果は `"リーダブルコード"` です。
- 同じように `book.year` なら `2012`、`book.isRead` なら `true` が取り出せます。
- 配列が番号 `[0]` で取り出すのに対し、オブジェクトは**名前（キー）**で取り出すのが違いです。

> **用語:** ドット記法（dot notation）＝ `オブジェクト.キー` の形で、オブジェクトの中の項目にアクセスする書き方。

---

5. 型に名前を付ける — `type` と `interface`

同じ形のオブジェクト（例: 本の情報）を何度も扱うとき、その「形」に名前を付けておくと便利で安全です。本書では主に `type` を使います。

> **▼ このコードがやること（先に日本語で）：** `type` というキーワードを使って「本1冊分の形（どんな項目を持つか）」に **`Book` という名前を付け**、その形に従って実際の本のデータを作ります。一度形を決めておくと、項目を書き忘れたときに「`title` が足りません」とエラーで教えてくれるので安全です。「`type 名前 = { ... }` でオブジェクトの形に名前を付けられる」という1点を押さえてください（各項目の意味はコード内コメント参照）。

コピー

```
// type : 型に名前を付けるキーワード / Book : 付ける名前（型名は大文字始まりが慣習）
type Book = {
  // id（識別番号）は文字列
  id: string;
  // title（タイトル）は文字列
  title: string;
  // author（著者）は文字列
  author: string;
  // status（読書状態）は文字列
  status: string;
  // isRead（読了したか）は真偽値
  isRead: boolean;
};
// ↑ これで「Book型 = id, title, author, status, isReadを持つオブジェクト」と定義できた

// 定義したBook型を使って変数を作る
// : Book → 「この変数はBook型ですよ」と宣言
const myBook: Book = {
  id: "1",
  title: "達人プログラマー",
  author: "David Thomas",
  status: "読書中",
  isRead: false,
};
// もしここで title を書き忘れると、「title が足りません」とエラーになる（型のおかげ）
```

▼ コードを1つずつ分解して解説

解説1: オブジェクトの「形」に名前を付ける（`type`）

コピー

```
// type : 型に名前を付けるキーワード / Book : 付ける名前（型名は大文字始まりが慣習）
type Book = {
  // id（識別番号）は文字列
  id: string;
  // title（タイトル）は文字列
  title: string;
  // author（著者）は文字列
  author: string;
  // status（読書状態）は文字列
  status: string;
  // isRead（読了したか）は真偽値
  isRead: boolean;
};
```

- `type` は「**型に名前を付ける**」キーワードです。`type 名前 = { ... };` の形で使います。
- `Book` が付けた名前（型名）です。型名は**大文字始まり**にするのが慣習です（変数名は小文字始まりだったのと対照的）。
- `{ ... }` の中は、**項目名と、その項目の型**の組を並べます。`id: string` なら「`id` という項目は文字列型」という意味です。
- これで「`Book` 型 ＝ `id`・`title`・`author`・`status`・`isRead` を持つオブジェクトの形」と定義できました。**実際のデータではなく「形（設計図）」を決めているだけ**です。

> **用語:** type（タイプ） ＝オブジェクトなどの「形」に名前を付けるキーワード。一度決めれば何度でも使い回せる。

解説2: 付けた型を使ってデータを作る（`: Book`）

コピー

```
// 定義したBook型を使って変数を作る
// : Book → 「この変数はBook型ですよ」と宣言
const myBook: Book = {
  id: "1",
  title: "達人プログラマー",
  author: "David Thomas",
  status: "読書中",
  isRead: false,
};
```

- `: Book` が型注釈で、「この `myBook` という変数は `Book` 型ですよ」と宣言しています。`: string` などと同じ位置（変数名の後ろ）に書きます。
- 中身の `{ ... }` は、`Book` 型で決めた**5つの項目をすべて、正しい型で**埋めています。

- もし `title` を書き忘れたり、`isRead: "yes"`（文字列）のように違う型を入れたりすると、**実行前にエラー**で教えてくれます。これが「形に名前を付けておく」最大のメリットです。

> **用語:** 型を使うことで「項目の書き忘れ」「型の取り違え」を実行前に防げる。これを「型安全」と呼ぶ。

> **`type` と `interface` の違い:** 同じく型に名前を付ける `interface`（インターフェース）というキーワードもあります。オブジェクトの形を定義する用途ではほぼ同じことができます。本書では一貫して `type` を使いますが、他の教材で `interface Book { ... }` を見ても「同じようなもの」と理解してください。

5.1「省略可能」を表す `?`

「あってもなくてもよい項目」は、キー名の後ろに `?`（クエスチョンマーク）を付けます。

> **▼ このコードがやること（先に日本語で）：** `Book` 型の `memo` という項目に `?` を付けて「あってもなくてもよい（省略可能な）項目」にし、`memo` を書いた場合と書かない場合の両方を作って見せます。`?` が付いている項目は、書いてもエラーにならず、書かなくてもエラーになりません。「キー名の後ろの `?` ＝ 省略してよい印」という1点だけ覚えればOKです。

コピー

```
type Book = {
  id: string;
  title: string;
  // memo? → memoは「あってもなくてもよい」項目（省略可能、optional）
  memo?: string;
};

// memoあり：OK
const book1: Book = { id: "1", title: "本A", memo: "面白い" };
// memoなし：これもOK（?のおかげ）
const book2: Book = { id: "2", title: "本B" };
```

---

6. 関数 — 処理に名前を付けてまとめる

関数（かんすう、function） は「一連の処理に名前を付けてまとめたもの」です。呼び出すと中の処理が動きます。

6.1 基本の書き方

> **▼ このコードがやること（先に日本語で）**：名前を1つ受け取って「こんにちは、〇〇さん」という挨拶の文を作って返す、`greet` という関数を作り、実際に呼び出して結果を表示します。関数に渡す材料を「引数（ひきすう）」、関数が返す結果を「戻り値」と呼びます。「`function 名前(引数) { ... return 結果 }` で処理に名前を付けてまとめ、あとから何度でも呼び出せる」という流れを押さえてください（各部分の意味はコード内コメント参照）。

コピー

```
// function : 関数を作るキーワード
// greet    : 関数名 / (name: string) : nameという文字列の引数を1つ受け取る / : string : 戻り値も文字列
function greet(name: string): string {
  // return : 結果を返す / + : 文字列をつなぐ（連結）
  return "こんにちは、" + name + "さん";
}

// greet関数を呼び出し、"鈴木"を渡す。戻り値が message に入る
const message = greet("鈴木");
// 結果: こんにちは、鈴木さん
console.log(message);
```

▼ コードを1つずつ分解して解説

解説1: 関数を定義する（引数と戻り値の型）

コピー

```
// function : 関数を作るキーワード
// greet    : 関数名 / (name: string) : nameという文字列の引数を1つ受け取る / : string : 戻り値も文字列
function greet(name: string): string {
  // return : 結果を返す / + : 文字列をつなぐ（連結）
  return "こんにちは、" + name + "さん";
}
```

- `function` は「関数を作る」キーワード、`greet` はその関数の名前です。
- `(name: string)` が**引数**の部分です。`name` が受け取る材料の名前、`: string` が「その材料は文字列」という型注釈です。
- `(...)` の後ろの `: string` は**戻り値の型**で、「この関数は文字列を返す」という意味です。引数の型と戻り値の型は書く場所が違うので注意します。
- `{ ... }` の中が関数の本体（処理）です。`return` は「この値を結果として返す」キーワード、`+` は文字列を**つなぐ（連結する）**記号です。`"こんにちは、" + name + "さん"` で1つの文章を組み立てています。

> **用語:** 引数（ひきすう / argument）＝関数に渡す材料。戻り値（return value）＝ `return` で返す結果。

解説2: 関数を呼び出して結果を受け取る

コピー

```
// greet関数を呼び出し、"鈴木"を渡す。戻り値が message に入る
const message = greet("鈴木");
// 結果: こんにちは、鈴木さん
console.log(message);
```

- `greet("鈴木")` が「関数を**呼び出す**」書き方です。関数名の後ろに `(...)` を付けて、中に渡す値を書きます。
- ここで渡している `"鈴木"` が、関数側の `name` に入ります。これを「引数を渡す」と言います。
- 関数が `return` で返した値（`"こんにちは、鈴木さん"`）が、`= message` で `message` という箱に入ります。
- 一度作った関数は、`greet("田中")` のように**何度でも呼び出せる**のが利点です。

> **用語:** 関数呼び出し（function call） = `関数名(引数)` の形で関数を実行し、戻り値を受け取ること。

> 「**引数（ひきすう、argument）**」と「**戻り値（return value）**」: - **引数**: 関数に渡す材料。`greet("鈴木")` の `"鈴木"` の部分。- **戻り値**: 関数が処理結果として返す値。`return` の後ろに書いた値。

6.2 アロー関数 — もう一つの書き方

React/React Nativeでは、`=>`（矢印のような記号）を使った**アロー関数**という短い書き方を非常によく使います。覚えておきましょう。

> **▼ このコードがやること（先に日本語で）**: さきほどと**まったく同じ挨拶の関数**を、`=>`（アロー＝矢印）を使った別の書き方「アロー関数」で書き直しています。動作は同じで、書き方の見た目が変わるだけです。React Nativeではこの `(引数) => { 処理 }` の形が主流なので、「`function` を使わなくても関数は作れる」と知っておいてください（記号の意味はコード内コメント参照）。

コピー

```
// const greet : 関数を変数に入れる形 / (name: string): string : 引数と戻り値の型 / => : アロー（矢印）
const greet = (name: string): string => {
  return "こんにちは、" + name + "さん";
};
// 上の function版とまったく同じ動作。React Nativeではこのアロー関数の形が主流
```

▼ コードを1つずつ分解して解説

解説1: 関数を変数に入れる（`const 名前 = ...`）

コピー

```
// const greet : 関数を変数に入れる形 / (name: string): string : 引数と戻り値の型 / => : アロー（矢印）
const greet = (name: string): string => {
```

- `const greet =` は「`greet` という変数に、これから書く関数を**入れる**」という形です。`function greet(...)` のように名前を直接書くのではなく、変数に代入するのがアロー関数の書き方です。
- `(name: string)` は引数、`: string` は戻り値の型で、ここは `function` 版とまったく同じです。
- `=>` が**アロー（矢印）**です。引数の `(...)` と関数本体の `{ }` の間に置いて「引数を受け取って処理する関数」を表します。

> **用語:** アロー関数（arrow function） = `(引数) => { 処理 }` の形で書く関数。`function` を使わずに関数を作れる。

解説2: 本体の処理（中身は `function` 版と同じ）

コピー

```
  return "こんにちは、" + name + "さん";
};
```

- `=>` の後ろの `{ ... }` が関数の本体で、`return` で結果を返すのは `function` 版と完全に同じです。
- つまり**動作は `function greet` 版とまったく同じ**で、書き方（見た目）だけが変わっています。
- アロー関数を変数に入れた場合、本体の `}` の後ろに**セミコロン `;` を付ける**点に注意します（`const ... = ...;` という代入文だからです）。

> **用語:** React / React Native では、この `const 名前 = (引数) => { ... }` のアロー関数の形が主流。慣れておくと読みやすい。

> **`=>` の読み方:** 「アロー（arrow＝矢印）」と読みます。`(引数) => { 処理 }` で「引数を受け取って処理する関数」を表します。一見とっつきにくいですが、慣れると簡潔で読みやすい書き方です。

---

7. よく使う配列の操作 — `map` と `filter`

アプリでは「本のリストを画面に並べる」「読了した本だけ抜き出す」といった操作を頻繁に行います。これに使うのが `map` と `filter` です。**第7章以降で必ず出てくる**ので、ここで基礎を押さえます。

7.1 `map` — 各要素を変換して新しい配列を作る

> **▼ このコードがやること（先に日本語で）**：数値の配列 `[1, 2, 3]` の**各要素を2倍した新しい配列** `[2, 4, 6]` を、`map`（マップ）という機能で作ります。`map` は「配列の1つ1つに同じ処理をして、その結果を集めた新しい配列を返す」働きをします。**元の配列は変わらず、新しい配列ができる**点を押さえてください。実際のアプリでは「本の配列」を「画面に表示する部品の配列」に変換するのに使います（処理の詳細はコード内コメント参照）。

コピー

```
// 数値の配列
const numbers = [1, 2, 3];

// map ： 各要素に処理をして新しい配列を作る
const doubled = numbers.map((n) => n * 2);
// .map(...)      ： 配列の各要素に対して、( )内の関数を順番に適用する
// (n) => n * 2   ： 「受け取った n を 2倍して返す」アロー関数。nには1,2,3が順に入る
// 結果: doubled は [2, 4, 6] になる（元のnumbersは変わらない）
```

▼ コードを1つずつ分解して解説

解説1: 元になる配列を用意する

コピー

```
// 数値の配列
const numbers = [1, 2, 3];
```

- `[1, 2, 3]` は数値が3つ並んだ配列です。これが「変換のもとになるデータ」です。
- 型注釈を書いていませんが、TypeScript が「右が数値の配列だから `number[]` 型だ」と**自動で推測**してくれます（型推論）。
- この `numbers` 自体は、このあと `map` を使っても**変化しません**（次の解説で重要になります）。

> **用語:** 型推論（type inference）＝代入された値から、TypeScript が型を自動で判断してくれる仕組み。

解説2: `map` で各要素を変換して新しい配列を作る

コピー

```
// map ： 各要素に処理をして新しい配列を作る
const doubled = numbers.map((n) => n * 2);
```

- `numbers.map(...)` の `.map` は「配列の**各要素1つ1つに、同じ処理をする**」メソッドです。配列の後ろにドットで付けて使います。
- `(n) => n * 2` は `map` に渡すアロー関数です。`n` には配列の要素が**順番に1つずつ**入ります（1回目は `1`、2回目は `2`、3回目は `3`）。
- `n * 2` で各要素を2倍した値を返し、その結果を集めた**新しい配列** `[2, 4, 6]` ができて `doubled` に入ります。
- **元の** `numbers` **は変わりません。** `map` は元の配列を壊さず、新しい配列を作って返すのが特徴です。

> **用語:** map（マップ）＝配列の各要素を変換し、その結果を集めた新しい配列を返すメソッド。一覧表示でよく使う。

> **React Nativeでの** `map` **の使いどころ:** 「本の配列」を「画面に表示する本カードの配列」に変換するのにこの `map` を使います。第7章で実際に登場します。

7.2 `filter` — 条件に合う要素だけ残す

> **▼ このコードがやること（先に日本語で）:** 本の配列の中から「**読了した本（** `isRead` **が** `true` **）だけ**」を抜き出して、新しい配列を作ります。これに使うのが `filter`（フィルター）で、「条件に合う要素だけを残す（ふるいにかける）」働きをします。`map` と同じく元の配列は変えず、条件に合うものだけ集めた新しい配列ができる、という点を押さえてください（`===` などの記号の意味はコード内コメント参照）。

コピー

```
const books = [
  { title: "本A", isRead: true },
  { title: "本B", isRead: false },
  { title: "本C", isRead: true },
];

// filter : 条件に合う要素だけ抜き出す
const readBooks = books.filter((b) => b.isRead === true);
// .filter(...)          : 各要素のうち、( )内の関数が true を返したものだけ残す
// (b) => b.isRead === true :  「その本の isRead が true か？」を判定する関数
// === :  「左右が厳密に等しいか」を判定する比較演算子（= が1つだと代入になるので注意）
// 結果: readBooks は 本A と 本C の2つだけの配列になる
```

▼ コードを1つずつ分解して解説

解説1: 絞り込みのもとになる配列を用意する

コピー

```
const books = [
  { title: "本A", isRead: true },
  { title: "本B", isRead: false },
  { title: "本C", isRead: true },
];
```

- `[ ... ]` の中に `{ ... }` のオブジェクトが3つ並んでいます。つまり「**オブジェクトの配列**」（本が3冊分のリスト）です。
- 各オブジェクトは `title`（タイトル）と `isRead`（読了したか）の2項目を持ちます。
- このうち `isRead` が `true` なのは「本A」と「本C」の2冊です。この2冊だけを次の `filter` で抜き出します。

> **用語:** オブジェクトの配列＝ `{ }` を要素に持つ配列。実際のアプリでは「本のリスト」などをこの形で扱う。

解説2: `filter` で条件に合う要素だけ残す

コピー

```
// filter : 条件に合う要素だけ抜き出す
const readBooks = books.filter((b) => b.isRead === true);
```

- `books.filter(...)` の `.filter` は「配列の中から**条件に合う要素だけを残す**（ふるいにかける）」メソッドです。
- `(b) => b.isRead === true` が判定の関数です。`b` には各本（オブジェクト）が順番に入り、`b.isRead === true` が `true` になった本**だけ**が残ります。
- `===`（イコール3つ）は「**左右が厳密に等しいか**」を判定する比較演算子です。`=`（1つ）は代入なので、判定と取り違えないよう注意します。
- 結果、`readBooks` には「本A」と「本C」の2つだけが入ります。`map` と同じく**元の `books` は変わりません。**

> **用語:** filter（フィルター）＝判定関数が `true` を返した要素だけを集めた新しい配列を返すメソッド。

> **`===`（イコール3つ）に注意:** `=`（1つ）は「代入（箱に入れる）」、`===`（3つ）は「等しいか判定する」です。初心者がよく間違えるポイントなので意識しましょう。「等しいかを聞きたいときは=を3つ」と覚えてください。

---

8. 条件分岐 `if` と 三項演算子

8.1 `if` 文 — 条件によって処理を分ける

> **▼ このコードがやること（先に日本語で）**：「もし状態が"読了"なら…、そうでなければ…」のように、**条件によって実行する処理を切り替える**書き方を見せています。`if（条件）{ ... } else { ... }` で、条件が成り立つ（`true`）ときは前の `{ }`、成り立たない（`false`）ときは `else` の `{ }` が動きます。「`if` で処理を枝分かれさせられる」という1点を押さえてください（`===` の意味はコード内コメント参照）。

コピー

```
const status = "読了";

// if ： 「もし(条件)なら」 / ( )内が条件 / === で等しいか判定
if (status === "読了") {
  // 条件が true のとき、この { } 内が実行される
  console.log("読み終わった本です");
// else ： 「そうでなければ」
} else {
  // 条件が false のとき、こちらが実行される
  console.log("まだ読んでいません");
}
// 結果: status が "読了" なので「読み終わった本です」が表示される
```

▼ コードを1つずつ分解して解説

解説1: 判定したい値を用意し、`if` で条件を書く

コピー

```
const status = "読了";

// if ： 「もし(条件)なら」 / ( )内が条件 / === で等しいか判定
if (status === "読了") {
  // 条件が true のとき、この { } 内が実行される
  console.log("読み終わった本です");
```

- まず `status` という変数に `"読了"` という文字列を入れています。これが判定の対象です。
- `if (...)` は「**もし（カッコの中の条件）が成り立つなら**」という意味です。カッコ `( )` の中に条件を書きます。
- `status === "読了"` は「`status` が `"読了"` と厳密に等しいか」を判定し、`true` か `false` を返します。ここでは `status` が `"読了"` なので `true` です。
- 条件が `true` のとき、直後の `{ ... }` の中（`console.log(...)`）が実行されます。

> **用語:** if 文＝条件が成り立つ（ `true` ）ときだけ `{ }` の中を実行する制御構文。

解説2: `else` で「そうでなければ」の処理を書く

コピー

```
// else : 「そうでなければ」
} else {
  // 条件が false のとき、こちらが実行される
  console.log("まだ読んでいません");
}
```

- `else` は「**条件が成り立たなかった（ `false` の）とき**」に動く部分です。`if` の `{ }` を閉じた直後に書きます。
- `else` の `{ ... }` の中は、条件が `false` のときだけ実行されます。`if` 側と `else` 側の**どちらか一方だけ**が必ず動きます。
- 今回は `status` が `"読了"` で条件が `true` なので、`else` 側の `"まだ読んでいません"` は実行されません。

> **用語:** else（エルス） ＝if の条件が成り立たなかったときに実行する部分。「そうでなければ」と読む。

8.2 三項演算子 — 短い条件分岐

React Nativeの画面コードでは、`if` 文の代わりに**三項演算子**という1行で書ける条件分岐をよく使います。

> **▼ このコードがやること（先に日本語で）:** さきほどの `if` と似た「条件による切り替え」を、**1行で書ける「三項演算子」**で行います。形は `条件 ? Aの値 : Bの値` で、「条件が `true` なら A、`false` なら B」を選んで返します。後の章の画面コードでは表示の出し分けにこの形を多用するので、「 `?` の前が条件、`?` と `:` の間が真のとき、`:` の後が偽のとき」と覚えてください。

コピー

```
const isRead = true;
const label = isRead ? "読了" : "未読";
// 条件 ? Aの値 : Bの値  という形
// isRead（条件）が true なら "読了"、false なら "未読" が label に入る
// ? の前が条件、? と : の間が「真のときの値」、: の後が「偽のときの値」
// 結果: label は "読了"
```

▼ コードを1つずつ分解して解説

解説1: 判定したい値を用意する

コピー

```
const isRead = true;
```

- `isRead` という変数に真偽値 `true` を入れています。これが三項演算子の「条件」に使う値です。
- 変数名を `is` で始めると「はい/いいえで答えられる値」だと分かりやすく、条件判定によく使われます。

> **用語:** 真偽値（boolean） = `true` / `false` のどちらか。条件判定の材料になる。

解説2: `条件 ? A : B` で1行で値を選ぶ

コピー

```
const label = isRead ? "読了" : "未読";
```

- これが**三項演算子**で、形は `条件 ? Aの値 : Bの値` です。`if` / `else` と似た分岐を**1行**で書けます。
- 読み方は「`?` の**前**が条件」「`?` と `:` の**間**が条件が `true` のときの値」「`:` の**後**が条件が `false` のときの値」です。
- ここでは `isRead`（条件）が `true` なので `"読了"` が選ばれ、`label` に入ります。もし `false` なら `"未読"` が入ります。
- `if` 文と違い、三項演算子は**1つの値を返す式**なので、`const label = ...` のように変数に直接代入できます。後の章の画面コード（JSX）で表示の出し分けに多用します。

> **用語:** 三項演算子（ternary operator） = `条件 ? A : B` の形で、条件に応じて2つの値のどちらかを選ぶ式。

> **なぜ三項演算子を使う？** React Nativeの画面コードの中（後の章で出てくるJSX）では、`if` 文がそのままでは書きにくい場面があります。そこで「条件 ? A : B」の三項演算子が画面の表示切り替えに重宝します。第4章以降で実際に使います。

---

9. ジェネリクス — 型に「中身の型」を渡すしくみ（`<...>`）

後の章では `useState<Book[]>(...)` や `Promise<Book[]>`、`useLocalSearchParams<{ id: string }>()` のように、**型名のうしろに山カッコ `<...>` が付く書き方**が頻繁に出てきます。これを **ジェネリクス**（generics） と呼びます。初出でいきなり出るとギョッとするので、ここで意味を押さえておきましょう。

ジェネリクスは「**入れ物の型に、"中に入る物の型"を渡す**」しくみです。身近なたとえで言うと、「箱」という型に「何を入れる箱か（みかん用／本用）」を指定するイメージです。

> **▼ このコードがやること（先に日本語で）**：配列の型 `string[]` を、`Array<string>` という別の書き方で表せること（どちらも「文字列の配列」で同じ意味）を見せています。`<...>`（山カッコ）の中に書いた型が「**中に入る物の型**」になります。「`Array<本の型>` なら本の配列」というように、**`<...>` の中は中身の型を教えているだけ**と分かれば、後の章で出てくる `useState<Book[]>` なども読めるようになります（自分で設計する必要は当面ありません）。

コピー

```
// 配列の型 string[] は、実は Array<string> とも書ける（同じ意味）
// Array       : 「配列」という"入れ物"の型
// <string>    : 「その配列の中身は文字列(string)ですよ」と中身の型を渡している
// string[] と全く同じ意味
const titles: Array<string> = ["本A", "本B"];

// 中身を Book にすれば「本の配列」になる
//  （Book型は第6章で定義します。「本1冊分の形」と思ってください）
// Book[] と同じ
// const books: Array<Book> = [...];
```

▼ コードを1つずつ分解して解説

解説1: `Array<string>` は `string[]` と同じ意味

コピー

```
// 配列の型 string[] は、実は Array<string> とも書ける（同じ意味）
// Array       : 「配列」という"入れ物"の型
// <string>    : 「その配列の中身は文字列(string)ですよ」と中身の型を渡している
// string[] と全く同じ意味
const titles: Array<string> = ["本A", "本B"];
```

- `Array` は「**配列という入れ物**」を表す型です。それ自体だけでは「何を入れる配列か」が決まっていません。
- `<string>`（山カッコの中の型）が、その入れ物に渡す「**中に入る物の型**」です。`Array<string>` で「中身は文字列の配列」になります。
- これは前に学んだ `string[]` と**まったく同じ意味**です。`型[]` という書き方と `Array<型>` という書き方の2通りがあるだけです。
- `<...>`（山カッコ）で型に「中身の型」を渡すこの仕組みを**ジェネリクス**と呼びます。

> **用語:** ジェネリクス（generics） = `入れ物の型<中身の型>` の形で、入れ物に「中に入る物の型」を渡す仕組み。

解説2: 中身の型を変えれば別の配列になる

コピー

```
// 中身を Book にすれば「本の配列」になる
//  （Book型は第6章で定義します。「本1冊分の形」と思ってください）
// Book[] と同じ
// const books: Array<Book> = [...];
```

- `<...>` の中を `Book` に変えて `Array<Book>` と書けば、「**本（Book型）の配列**」という意味になります。
- これも `Book[]` という書き方と同じです。同じ `Array` という入れ物でも、`<string>` か `<Book>` かで「中に入る物」が変わります。
- このように「既にある入れ物（`Array`・`useState`・`Promise` など）に、後から中身の型を渡すだけ」と考えれば、後の章の `useState<Book[]>` などもこわくありません。**自分でジェネリクスを設計する必要は当面ありません。**

> **用語:** `Array<Book>` = `Book[]` 。どちらも「本の配列」を表す。中身の型を変えれば別の配列の型になる。

- `<...>` の中に書いた型が「**中身の型**」になります。`Array<string>` は「文字列の配列」、`Array<Book>` は「本の配列」です。
- 同じ「配列」という入れ物でも、`<string>` か `<Book>` かで「中に何が入るか」が変わります。これがジェネリクス（＝汎用的に、中身の型を後から指定できる）の便利さです。

> **後の章での登場例（今は眺めるだけでOK）:** - `useState<Book[]>([])` …「**Bookの配列**を持つstate」を作る（第7章）。`<Book[]>` で中身の型を指定。- `Promise<Book[]>` …「**最終的にBookの配列を返す**非同期処理」の型（下の第9.5節・第6章）。- `useLocalSearchParams<{ id: string }>()` …「`id` という文字列を受け取る」と中身の型を指定（第8章）。
>
> いずれも「`<...>` の中は"中身の型"を教えているんだな」と分かれば読めます。**自分でジェネリクスを設計する必要は当面ありません。**「既にある入れ物に中身の型を渡すだけ」と考えてください。

9.5 非同期処理 — `async` / `await` と `Promise`

アプリは「サーバーからデータを取ってくる」という**時間のかかる処理**を行います。データが届くまで他の処理を止めずに待つ仕組みが `async`（**エイシンク**）/ `await`（**アウェイト**） です。**第6・7章でSupabaseからデータを取得するときに必ず使**

うので、概念をしっかり掴んでおきましょう。

> **▼ このコードがやること（先に日本語で）**：「サーバーから本のデータを取ってくる」ような**時間のかかる処理を、結果が届くまで待ってから次に進む**書き方を見せています。関数に `async`（エイシンク）という目印を付けると、その中で `await`（アウェイト）＝「終わるまでここで待つ」が使えます。**`await` を付け忘れると、データが届く前に先へ進んでしまいバグになる**ので、「時間のかかる処理の前には `await` 」とセットで覚えてください（詳細はコード内コメントと後続の解説にあります）。

コピー

```
// async ： 「この関数の中には"待つ処理"が含まれる」という目印を付けるキーワード
async function loadBooks() {
  // await ： 「この処理が終わるまで、ここで待つ」という指示
  // fetchBooksFromServer() はサーバーからデータを取る（時間がかかる）関数だと考えてください
  // データが届くまで待ってから、結果を books に入れる
  const books = await fetchBooksFromServer();
  // 届いたデータを表示
  console.log(books);
}
```

▼ コードを1つずつ分解して解説

解説1: `async` を付けて「待つ処理を含む関数」にする

コピー

```
// async ： 「この関数の中には"待つ処理"が含まれる」という目印を付けるキーワード
async function loadBooks() {
```

- `async`（エイシンク）は関数の前に付ける**目印**です。「この関数の中には"終わるまで待つ処理"が含まれますよ」という宣言です。
- `async` を付けると、その関数の中で `await`（次の解説）が使えるようになります。逆に `async` の無い関数の中では `await` は使えません。だから2つはセットで登場します。
- `async` を付けた関数の**戻り値は必ず `Promise<...>` 型**になります（後述）。

> **用語**: async（エイシンク）＝関数を「非同期関数」にする目印。中で `await` を使えるようにする。

解説2: `await` で結果が届くまで待つ

コピー

```
// await ： 「この処理が終わるまで、ここで待つ」という指示
// fetchBooksFromServer() はサーバーからデータを取る（時間がかかる）関数だと考えてください
// データが届くまで待ってから、結果を books に入れる
const books = await fetchBooksFromServer();
// 届いたデータを表示
console.log(books);
```

- `fetchBooksFromServer()` は「サーバーからデータを取ってくる」**時間のかかる処理**だと考えてください。呼んだ瞬間には結果がまだありません。
- その前に付いた `await` （アウェイト）が「**この処理が終わる（データが届く）まで、ここで待つ**」という指示です。
- 待った後、届いたデータが `= books` で `books` に入ります。だから次の `console.log(books)` では、ちゃんと中身が入った状態で表示できます。
- もし `await` を**付け忘れる**と、データが届く前に次の行へ進んでしまい、`books` が空っぽのまま表示されるバグになります。「**時間のかかる処理の前には `await` **」とセットで覚えます。

> **用語:** await（アウェイト）＝結果が届くまでその行で待つ指示。`async` 関数の中でのみ使える。

---

① **`await` を付けないとどうなる？** `await` は「結果が届くまでこの行で待つ」という指示です。もし `await` を**付け忘れる**と、プログラムは「データが届くのを待たずに次の行へ進んで」しまいます。すると `books` にはまだデータが入っておらず、空っぽや「処理中の状態」を表示してしまう、というバグになります。「**時間のかかる処理の前には `await` **」とセットで覚えてください。なお `await` は `async` を付けた関数の中でしか使えません（だから2つはセットで登場します）。

② **`Promise` （プロミス）とは？**「時間のかかる処理」は、呼んだ瞬間には結果がまだありません。その「**結果は後で渡すよ、という"引換券"**」のことを `Promise` と呼びます。レストランで料理を注文したときの「番号札」のイメージです。

> **Promiseの3つの状態（やさしく）**：Promiseは次のどれかの状態を持ちます。- **待機中（pending）** … まだ料理ができていない（処理の途中）。- **成功（fulfilled）** … 料理ができて受け取れた（データが届いた）。- **失敗（rejected）** … 料理が作れなかった（通信エラーなど）。
>
> `await` は「この番号札（Promise）が"成功"か"失敗"になるまで待って、中身（料理＝データ）を受け取る」働きをします。失敗したときの受け止め方が、第7章で学ぶ `try / catch` です。

③ **`Promise<Book[]>` の読み方（ジェネリクス再登場）** 第6章では `function fetchBooks(): Promise<Book[]>` のような書き方が出ます。これは第9節のジェネリクスです。「 `Promise<Book[]>` 」は「**最終的に Book の配列（ `Book[]` ）を渡してくれる引換券**」という意味。`async` を付けた関数の戻り値は、必ずこの `Promise<...>` の形になります（中身が無いなら `Promise<void>` 。`void` は「返す値なし」の型）。

身近な例え（再）：カップ麺にお湯を入れて「3分待つ」のが `await`、その「3分後にできあがる」という約束が `Promise` です。`async` は「この調理には待ち時間がありますよ」という札。待っている間に他のこともできるのが非同期（asynchronous＝同時並行的）の良さです。今は「サーバー通信には async/await を使い、戻り値は Promise になる」と分かればOKです。

発展: アプリでは使っていない重要なTypeScript機能

ここまでで、後の章のコードを読むのに必要な文法は一通りそろいました。この発展セクションでは、本書のアプリ作りには直接出てこないものの、**実際の現場ではよく使う重要なTypeScriptの機能**を、独立した小さな例で紹介します。「今すぐ全部覚えなくてOK、こういうものがあると知っておく」くらいの気持ちで読んでください。

発展1: ユニオン型・リテラル型 — 「このどれか」を表す

▼ このコードがやること（先に日本語で）：変数に入れてよい値を「**このいくつかの中のどれか**」に限定する書き方を見せています。`"読了" | "読書中" | "未読"` のように、`|`（縦棒）で候補を並べると「この3つのうちのどれか1つしか入れられない」型になります。これを使うと、決まった値しか入らないことを実行前に保証できます。「`|` は"または"、候補を `|` で並べると限定できる」という1点を押さえてください（各部分の意味はコード内コメント参照）。

コピー

```
// 「文字列ならなんでも」ではなく、決まった3つの言葉だけを許す型を作る
// "読了" | "読書中" | "未読" : |（縦棒）で候補を並べる。「このどれか1つ」という意味
// リテラル型をユニオンでつないだ型
type Status = "読了" | "読書中" | "未読";

// 候補の中にあるのでOK
const s1: Status = "読了";
// ← 候補にない言葉なので、実行前にエラーになる
// const s2: Status = "積読";

// 型どうしを | でつなぐこともできる（文字列または数値、など）
// id には文字列か数値のどちらかを入れてよい
let id: string | number;
// 文字列：OK
id = "abc";
// 数値：OK
id = 123;
```

▼ コードを1つずつ分解して解説

解説1: 決まった言葉だけを許す（リテラル型 + ユニオン）

コピー

```
// リテラル型をユニオンでつないだ型
type Status = "読了" | "読書中" | "未読";

// 候補の中にあるのでOK
const s1: Status = "読了";
// ← 候補にない言葉なので、実行前にエラーになる
// const s2: Status = "積読";
```

- `"読了"` のように**特定の値そのもの**を型にしたものを「**リテラル型**」と呼びます。`string`（文字列ならなんでも）よりも厳しく、「まさにこの言葉だけ」を表します。
- それを `|`（縦棒）でつないだ `"読了" | "読書中" | "未読"` が「**このどれか1つ**」という型です。この `|` でつなぐ書き方を「**ユニオン型**」と呼びます。
- `Status` 型の変数には、この3つの言葉以外は入れられません。`"積読"` のように候補にない値を入れると、**実行前にエラー**で教えてくれます。打ち間違いを防げるのが利点です。

> **用語:** リテラル型＝ `"読了"` のように「特定の値そのもの」を表す型。ユニオン型＝ `A | B` の形で「AまたはB（のどれか）」を表す型。

解説2: 型どうしを `|` でつなぐ（文字列または数値）

コピー

```
// id には文字列か数値のどちらかを入れてよい
let id: string | number;
// 文字列：OK
id = "abc";
// 数値：OK
id = 123;
```

- ユニオン型は、`string | number` のように**型どうし**をつなぐこともできます。「文字列**または**数値のどちらか」という意味です。
- そのため `id` には文字列 `"abc"` を入れても、数値 `123` を入れてもエラーになりません。
- ただし `true`（真偽値）など、候補にない型を入れようとするとエラーになります。「許す型を `|` で並べる」と覚えてください。

> **用語:** `string | number` ＝「文字列または数値」。`|` は型の世界では「または」の意味になる。

発展2: Utility Types — 既存の型から新しい型を作る道具

> **▼ このコードがやること（先に日本語で）**：すでにある型（ここでは本の型 `Book` ）をもとに、「**全項目を省略可能にした型**」「**一部の項目だけ取り出した型**」などを、TypeScriptが用意してくれた便利な道具で**自動で作って**います。同じ項目を何度も手書きしなくて済むのが利点です。`Partial` ・ `Pick` ・ `Omit` ・ `Required` ・ `Record` という5つの名前と「それぞれ何をするか」をざっくり知っておけばOKです（各道具の意味はコード内コメント参照）。

コピー

```
// もとになる型（本1冊分の形）
type Book = {
  id: string;
  title: string;
  author: string;
  isRead: boolean;
};

// Partial<T> : Tの全項目を「省略可能(?)」にした型を作る
type PartialBook = Partial<Book>;
// → { id?: string; title?: string; author?: string; isRead?: boolean; } と同じ
// 一部だけでOK
const draft: PartialBook = { title: "下書き" };

// Pick<T, "キー名"> : Tから指定した項目「だけ」取り出した型を作る
type BookPreview = Pick<Book, "id" | "title">;
// → { id: string; title: string; } と同じ
const preview: BookPreview = { id: "1", title: "本A" };

// Omit<T, "キー名"> : Tから指定した項目「を除いた」型を作る
type BookWithoutId = Omit<Book, "id">;
// → { title: string; author: string; isRead: boolean; } と同じ（idが消える）

// Required<T> : Tの全項目を「必須（?を外す）」にした型を作る（Partialの逆）
type FullBook = Required<PartialBook>;
// → 全項目が省略不可に戻る

// Record<キーの型, 値の型> : 「キーと値の組」の型をまとめて作る
//  「名前(文字列)→点数(数値)」の対応表
type ScoreByName = Record<string, number>;
const scores: ScoreByName = { 鈴木: 80, 田中: 95 };
```

▼ コードを1つずつ分解して解説

解説1: もとになる型を用意する

コピー

```
type Book = {
  id: string;
  title: string;
  author: string;
  isRead: boolean;
};
```

- まず「本1冊分の形」を `Book` 型として定義しています。これは第5節で学んだ `type` と同じ書き方です。
- このあと紹介する道具は、すべて**この `Book` をもとに別の型を作る**ものです。もとの型は1つ書いておけば、そこから何種類もの型を自動で生み出せます。

> **用語:** Utility Types（ユーティリティ型） ＝既存の型から新しい型を作るために、TypeScriptが標準で用意してくれている便利な道具。

解説2: `Partial` で全項目を省略可能にする

コピー

```
type PartialBook = Partial<Book>;
// 一部だけでOK
const draft: PartialBook = { title: "下書き" };
```

- `Partial<Book>` は「`Book` の**全項目に `?` を付けた型**」を作ります。第5節で学んだ `?`（省略可能）を、全項目にまとめて付けるイメージです。
- ジェネリクス（第9節）と同じ `<...>` の中に「もとの型」を渡します。`Partial<Book>` の `<Book>` がそれです。
- そのため `draft` は `title` だけを書いてもエラーになりません。「入力途中の下書きデータ」などを表すのに便利です。

> **用語:** `Partial<T>` ＝ 型 `T` の全項目を省略可能（`?` 付き）にした型を作る道具。

解説3: `Pick` で一部の項目だけ取り出す

コピー

```
type BookPreview = Pick<Book, "id" | "title">;
const preview: BookPreview = { id: "1", title: "本A" };
```

- `Pick<Book, "id" | "title">` は「`Book` の中から `id` と `title` の**2項目だけを取り出した型**」を作ります。

- 取り出したい項目名を、`"id" | "title"` のように**ユニオン型**（発展1）で並べて指定します。
- 結果の `BookPreview` 型は `id` と `title` しか持たないので、一覧表示で「タイトルだけ見せたい」ようなときに役立ちます。

> **用語:** `Pick<T, K>` ＝ 型 `T` から指定した項目 `K` だけを取り出した型を作る道具。

解説4: `Omit` で一部の項目を除く

コピー

```
type BookWithoutId = Omit<Book, "id">;
// → { title: string; author: string; isRead: boolean; } と同じ（idが消える）
```

- `Omit<Book, "id">` は `Pick` の**逆**で、「`Book` から `id` を**除いた**型」を作ります。
- 残りの `title`・`author`・`isRead` を持つ型になります。「IDはサーバーが自動で付けるので、新規作成時のデータからは除きたい」といった場面で便利です。
- 除きたい項目が複数あるときは、`Omit<Book, "id" | "isRead">` のようにユニオン型で並べます。

> **用語:** `Omit<T, K>` ＝ 型 `T` から指定した項目 `K` を除いた型を作る道具（`Pick` の逆）。

解説5: `Required` で全項目を必須に戻す

コピー

```
type FullBook = Required<PartialBook>;
// → 全項目が省略不可に戻る
```

- `Required<T>` は `Partial` の**逆**で、「全項目の `?` を外して**必須にした型**」を作ります。
- ここでは、さきほど全項目を省略可能にした `PartialBook` を、`Required` でもう一度すべて必須に戻しています。
- 「下書き段階では省略可、保存時にはすべて埋まっていてほしい」というように、`Partial` と `Required` をセットで使うことがあります。

> **用語:** `Required<T>` ＝ 型 `T` の全項目を必須（`?` なし）にした型を作る道具（`Partial` の逆）。

解説6: `Record` で「キーと値の対応表」を作る

コピー

```
// 「名前(文字列)→点数(数値)」の対応表
type ScoreByName = Record<string, number>;
const scores: ScoreByName = { 鈴木: 80, 田中: 95 };
```

- `Record<キーの型, 値の型>` は「**キーと値の組をまとめた型**」を作ります。`<...>` に2つの型を渡すのが特徴です。
- `Record<string, number>` は「**キーは文字列、値は数値**」という意味で、`{ 鈴木: 80, 田中: 95 }` のような「名前→点数」の対応表を表せます。
- 項目名が事前に決まっていない「辞書のようなデータ」を扱うときに便利です。

> **用語:** `Record<K, V>` ＝ キーの型が `K`、値の型が `V` の「対応表（辞書）」の型を作る道具。

発展3: enum（列挙型） — 名前付きの定数をまとめる

> **▼ このコードがやること（先に日本語で）:** 「読書の状態」のような**決まった選択肢のグループ**に、`enum`（イーナム）という仕組みで名前を付けてまとめています。`Status.Read` のように「グループ名.選択肢名」で値を取り出せて、打ち間違いを防げます。発展1のリテラル型と似た目的ですが、こちらは「定数をひとまとめにする」書き方です。「`enum グループ名 { 選択肢1, 選択肢2 }` で選択肢をまとめられる」という形を押さえてください（詳細はコード内コメント参照）。

コピー

```
// enum : 関連する定数（選択肢）をひとまとめにするキーワード
// Status : グループ名（型名と同じく大文字始まりが慣習）
enum Status {
  // 未読   （何も指定しないと自動で 0 が割り当てられる）
  Unread,
  // 読書中（自動で 1）
  Reading,
  // 読了   （自動で 2）
  Read,
}

// 「Statusグループの Reading」を取り出す
const current: Status = Status.Reading;
// 結果: 1（Readingに割り当てられた番号）
console.log(current);

// 文字列を割り当てることもできる（こちらの方が読みやすく、よく使われる）
enum StatusLabel {
  Unread = "未読",
  Reading = "読書中",
  Read = "読了",
}
// 結果: "読了"
console.log(StatusLabel.Read);
```

▼ コードを1つずつ分解して解説

解説1: 選択肢をひとまとめにする（`enum`）

コピー

```
enum Status {
  // 未読   （何も指定しないと自動で 0 が割り当てられる）
  Unread,
  // 読書中（自動で 1）
  Reading,
  // 読了   （自動で 2）
  Read,
}

// 「Statusグループの Reading」を取り出す
const current: Status = Status.Reading;
// 結果: 1（Readingに割り当てられた番号）
console.log(current);
```

- `enum` は「**関連する選択肢（定数）をひとまとめにする**」キーワードです。`enum グループ名 { 選択肢1, 選択肢2, ... }` の形で書きます。
- 中の選択肢に何も指定しないと、上から順に **0, 1, 2... と番号が自動で割り当てられます**。だから `Status.Reading` は `1` になります。
- 値を取り出すときは `Status.Reading` のように「**グループ名.選択肢名**」と書きます。`.`（ドット）はオブジェクトのときと同じ使い方です。
- 数字を直接書く代わりに名前で扱えるので、コードが読みやすくなり、打ち間違いも防げます。

> **用語:** enum（イーナム / 列挙型）＝関連する定数（選択肢）に名前を付けてひとまとめにする仕組み。

解説2: 文字列を割り当てる（実用ではこちらが多い）

コピー

```
enum StatusLabel {
  Unread = "未読",
  Reading = "読書中",
  Read = "読了",
}
// 結果: "読了"
console.log(StatusLabel.Read);
```

- 各選択肢に `= "未読"` のように**値を明示**することもできます。ここでは番号ではなく文字列を割り当てています。
- `StatusLabel.Read` を表示すると `"読了"` が出ます。番号より意味が分かりやすいので、実際の現場ではこの**文字列を割り当てる書き方**がよく使われます。
- なお、発展1のリテラル型（`"読了" | "読書中" | "未読"`）でも似たことができます。近年のReact開発ではリテラル型を好む人も多く、両方とも見かける、と知っておけば十分です。

> **用語:** 文字列enum＝各選択肢に文字列を割り当てた `enum`。番号より意味が読み取りやすい。

発展4: 型ガード — 実行中に「今この値は何の型か」を見分ける

> **▼ このコードがやること（先に日本語で）**：ユニオン型（発展1）などで「文字列か数値のどちらか」のような値を扱うとき、**実行中に「今は実際どちらなのか」を調べて、安全に処理を分ける**書き方を見せています。`typeof`・`in`・`x is T` という3つの調べ方を紹介します。これらを使うと、TypeScriptが「このブロックの中ではこの型だ」と分かってくれて、安全にその型として扱えます。「値の種類を確かめてから使う」という考え方を押さえてください（各記号の意味はコード内コメント参照）。

コピー

```
// (1) typeof : 値が「文字列か数値か」などの基本の種類を調べる
function describe(value: string | number): string {
  // ← この { } の中では value は string として扱える
  if (typeof value === "string") {
    // 文字列だけが持つ .length を安全に使える
    return "文字列の長さ: " + value.length;
  } else {
    // ここでは value は number として扱える
    return "数値を2倍: " + value * 2;
  }
}

// (2) in : オブジェクトが「そのキーを持っているか」で見分ける
type Dog = { bark: () => void };
type Cat = { meow: () => void };
function speak(animal: Dog | Cat): void {
  // bark というキーを持つ？ → 持つなら Dog
  if ("bark" in animal) {
    animal.bark();
  } else {
    animal.meow();
  }
}

// (3) 型述語 x is T : 「この関数が true を返したら、x は T 型だ」と教える自作の判定関数
// 戻り値の型を「x is string」と書くのがポイント
function isString(x: unknown): x is string {
  return typeof x === "string";
}
const data: unknown = "こんにちは";
if (isString(data)) {
  // ← ここでは data は string として扱える
  console.log(data.toUpperCase());
}
```

▼ コードを1つずつ分解して解説

解説1: `typeof` で基本の種類を見分ける

コピー

```
function describe(value: string | number): string {
  // ← この { } の中では value は string として扱える
  if (typeof value === "string") {
    // 文字列だけが持つ .length を安全に使える
    return "文字列の長さ: " + value.length;
  } else {
    // ここでは value は number として扱える
    return "数値を2倍: " + value * 2;
  }
}
```

- 引数 `value` は `string | number`（文字列または数値）です。このままでは「文字列専用の機能」も「数値専用の機能」も安全には使えません。
- `typeof value === "string"` は「**`value` の種類が文字列か？**」を調べる書き方です。`typeof` は値の基本の種類（`"string"`・`"number"`・`"boolean"` など）を文字列で返します。
- この `if` が `true` のブロックの中では、TypeScriptが「`value` は文字列だ」と理解してくれるので、文字列だけが持つ `.length`（文字数）を安全に使えます。`else` 側では数値として `* 2` できます。
- このように「種類を確かめてから使う」ことで、間違った操作を防ぎます。これを**型ガード**と呼びます。

> **用語:** 型ガード（type guard）＝実行中に値の型を見分けて、そのブロック内でその型として安全に扱えるようにする仕組み。`typeof` はその代表的な手段。

解説2: `in` でオブジェクトのキーの有無を見分ける

コピー

```
function speak(animal: Dog | Cat): void {
  // bark というキーを持つ？ → 持つなら Dog
  if ("bark" in animal) {
    animal.bark();
  } else {
    animal.meow();
  }
}
```

- `animal` は `Dog | Cat`（犬または猫）です。犬は `bark`、猫は `meow` という別々の機能を持っています。
- `"bark" in animal` は「`animal` が **`bark` というキー（項目）を持っているか**」を調べる書き方です。`in` はオブジェクトにそのキーがあるかを判定します。
- `bark` を持つのは `Dog` だけなので、`true` のブロックの中では `animal` は `Dog` だと分かり、`animal.bark()` を安全に呼べます。`else` 側では `Cat` として `meow()` を呼びます。

- `typeof` では区別できない「オブジェクトどうしの違い」を見分けたいときに `in` が役立ちます。

> **用語:** `in` 演算子＝`"キー名" in オブジェクト` の形で、そのオブジェクトが特定のキーを持つか判定する。型ガードに使える。

解説3: 型述語 `x is T` で自作の判定関数を作る

コピー

```
// 戻り値の型を「x is string」と書くのがポイント
function isString(x: unknown): x is string {
  return typeof x === "string";
}
const data: unknown = "こんにちは";
if (isString(data)) {
  // ← ここでは data は string として扱える
  console.log(data.toUpperCase());
}
```

- `unknown`（アンノウン）は「**何の型か分からない値**」を表す型です。安全のため、そのままでは文字列の機能などを使えません。
- `isString` は「渡された `x` が文字列かどうか」を判定する自作の関数です。注目は戻り値の型 `x is string` で、これを「**型述語（かたじゅつご）**」と呼びます。「この関数が `true` を返したら、`x` は `string` 型だと思ってよい」とTypeScriptに教えています。
- そのため `if (isString(data))` の中では、TypeScriptが `data` を文字列だと理解し、文字列専用の `.toUpperCase()`（大文字化）を安全に使えます。
- `typeof` や `in` だけでは表せない「複雑な判定」を自分で関数にまとめたいときに、この `x is T` を使います。

> **用語:** 型述語（type predicate）＝ `引数 is 型` の形の戻り値の型。`true` を返したとき「その引数はその型だ」とTypeScriptに教える。`unknown` ＝「何の型か不明」を表す型。

発展5: `as const` — 値を「変わらない・厳密な型」として固定する

> **▼ このコードがやること（先に日本語で）:** 値の後ろに `as const`（アズ・コンスト）を付けると、その値を「**これ以上変えない、まさにこの値**」としてTypeScriptに認識させられます。これを付けないと「ただの文字列」「ただの配列」と緩く扱われますが、付けると発展1のリテラル型のように**厳密な型**になります。「決まった選択肢の一覧を `as const` で固定する」という使い方を押さえてください（違いはコード内コメント参照）。

コピー

```
// as const を付けない場合：ただの string と推論される（緩い）
// 型は string（後で別の文字列にも変えうる扱い）
const status1 = "読了";

// as const を付ける場合：「まさに "読了" 」というリテラル型に固定される（厳密）
// 型は "読了"（この値そのもの）
const status2 = "読了" as const;

// 配列やオブジェクトに付けると、中身も丸ごと「変更不可・厳密」になる
const statuses = ["未読", "読書中", "読了"] as const;
// → 型は readonly ["未読", "読書中", "読了"]（中身を書き換えられない、厳密な並び）

// この一覧から「どれか」を表す型を取り出せる（発展1のユニオン型になる）
// "未読" | "読書中" | "読了"
type Status = (typeof statuses)[number];
```

▼ コードを1つずつ分解して解説

解説1: `as const` の有無で型の厳しさが変わる

コピー

```
// 型は string（後で別の文字列にも変えうる扱い）
const status1 = "読了";
// 型は "読了"（この値そのもの）
const status2 = "読了" as const;
```

- `const status1 = "読了"` だけだと、TypeScriptは型を `string`（文字列ならなんでも）と緩く推論します。
- 値の後ろに `as const` を付けると、「**まさに `"読了"` という値**」という厳密なリテラル型（発展1）に固定されます。
- 「この値はこの先ずっとこのままで、別の文字列に変わることはない」とTypeScriptに伝えるのが `as const` です。

> **用語:** `as const`（アズ・コンスト）＝値を「変更しない・まさにこの値」として、厳密なリテラル型に固定する書き方。

解説2: 配列に付けて「選択肢の一覧」を固定する

コピー

```
const statuses = ["未読", "読書中", "読了"] as const;
// → 型は readonly ["未読", "読書中", "読了"]（中身を書き換えられない、厳密な並び）

// "未読" | "読書中" | "読了"
type Status = (typeof statuses)[number];
```

- 配列に `as const` を付けると、**中身も丸ごと「書き換え不可（readonly）」で「厳密な値」**になります。`statuses` は「この3つの言葉が、この順で並んだ変えられないリスト」になります。
- `(typeof statuses)[number]` は少し高度ですが、「この配列の**どれか1つの要素の型**」を取り出す書き方です。結果は `"未読" | "読書中" | "読了"` というユニオン型（発展1）になります。
- これにより「**選択肢の一覧（配列）を1か所に書くだけで、そこから型も自動で作れる**」ようになります。選択肢を1つ追加すれば型も自動で増えるので、二重管理がなくなり便利です。

> **用語:** `readonly`（リードオンリー）＝「読み取り専用＝中身を書き換えられない」という印。`as const` を配列に付けると自動で付く。

発展6: オプショナルチェイニング `?.` と Null合体演算子 `??`

> **▼ このコードがやること（先に日本語で）:** 「**あるかもしれないし、無いかもしれない値**」を、エラーで止まらずに安全に扱う2つの記号を紹介します。`?.`（オプショナルチェイニング）は「もし手前が無ければ、そこで止めて `undefined` を返す」、`??`（Null合体演算子）は「もし左が無ければ、右の値を代わりに使う」働きです。この2つはセットで「値が無いときも安全に進める」ために使います（各記号の意味はコード内コメント参照）。

コピー

```
type User = {
  name: string;
  // address は省略可能（無いかもしれない）
  address?: {
    // city も省略可能
    city?: string;
  };
};

// address を持たないユーザー
const user: User = { name: "鈴木" };

// ?. : 手前が null/undefined なら、そこで止めて undefined を返す（エラーにしない）
// address が無いので、city は undefined（エラーにならない）
const city = user.address?.city;
// もし普通に user.address.city と書くと、address が無いとき実行時エラーになる

// ?? : 左が null/undefined のときだけ、右の値を使う
// city が無いので "未設定" になる
const displayCity = user.address?.city ?? "未設定";
// 結果: "未設定"
console.log(displayCity);
```

▼ コードを1つずつ分解して解説

解説1: `?.` で「無いかもしれない値」を安全にたどる

コピー

```
// address を持たないユーザー
const user: User = { name: "鈴木" };

// address が無いので、city は undefined（エラーにならない）
const city = user.address?.city;
// もし普通に user.address.city と書くと、address が無いとき実行時エラーになる
```

- `user` は `address`（住所）を持っていません。`User` 型で `address?` と省略可能にしているからです。
- 普通に `user.address.city` と書くと、`address` が無い（`undefined`）のにさらに `.city` をたどろうとして、**実行時にエラーで止まって**しまいます。
- `user.address?.city` の `?.`（オプショナルチェイニング）は「**もし手前（`address`）が `null` か `undefined` なら、そこで止めて `undefined` を返す**」働きです。エラーで止まらず、安全に `undefined` を受け取れます。
- 「無いかもしれない項目をたどるときは `.` の代わりに `?.`」と覚えてください。

> **用語:** オプショナルチェイニング（ `?.` ）＝手前の値が `null` / `undefined` なら、その先をたどらず `undefined` を返す記号。エラーを防ぐ。

解説2: `??` で「無いとき用の代わりの値」を指定する

コピー

```
// city が無いので "未設定" になる
const displayCity = user.address?.city ?? "未設定";
// 結果: "未設定"
console.log(displayCity);
```

- `user.address?.city` は（前の解説のとおり） `undefined` になります。そのままでは画面に何も表示できません。
- `?? "未設定"` の `??` （Null合体演算子）は「**左の値が `null` か `undefined` のときだけ、右の値を代わりに使う**」働きです。
- ここでは左が `undefined` なので、右の `"未設定"` が選ばれ、`displayCity` は `"未設定"` になります。値があればその値がそのまま使われます。
- `?.` （安全にたどる）と `??` （無いとき用の値を用意する）はセットで使うことが多く、「値が無くても落ちない・ちゃんと表示できる」コードになります。

> **用語:** Null合体演算子（ `??` ）＝ `左 ?? 右` で、左が `null` / `undefined` のときだけ右の値を使う記号。「無いとき用の初期値」を指定するのに使う。

発展7: ジェネリック関数 — どんな型でも扱える「使い回せる関数」

> **▼ このコードがやること（先に日本語で）:** 第9節のジェネリクス（ `<...>` ）を、**関数**に応用しています。受け取った値が文字列でも数値でも本でも、その**型を保ったまま**処理できる「使い回せる関数」を作ります。ここでは「受け取った値を1個だけ入れた配列にして返す」関数 `wrap` を例にします。`<T>` の `T` は「**型の入れ物（あとから決まる型の名前）**」で、呼び出したときの値に合わせて中身が決まります。「 `<T>` で型を1つ受け取り、その型を使い回す」という考え方を押さえてください（詳細はコード内コメント参照）。

コピー

```
// <T> : 「これから決まる型」に T という仮の名前を付ける（型の入れ物）
// (x: T) : 受け取る値の型は T / : T[] : 戻り値は「Tの配列」
function wrap<T>(x: T): T[] {
  // 受け取った値を1個だけ入れた配列にして返す
  return [x];
}

// "本A" は文字列 → T は string と決まり、戻り値は string[]
const a = wrap("本A");
// 123 は数値   → T は number と決まり、戻り値は number[]
const b = wrap(123);
// a の型は string[]、b の型は number[] と、渡した値に合わせて自動で決まる

// 結果: ["本A"]
console.log(a);
// 結果: [123]
console.log(b);
```

▼ コードを1つずつ分解して解説

解説1: `<T>` で「あとから決まる型」を受け取る

コピー

```
function wrap<T>(x: T): T[] {
  // 受け取った値を1個だけ入れた配列にして返す
  return [x];
}
```

- 関数名の直後の `<T>` が、この関数で使う「**型の入れ物**」です。`T` は仮の名前で、「これから（呼び出すときに）決まる型」を表します（`T` は Type の頭文字。名前は自由ですが慣習でよく `T` を使います）。
- 引数 `(x: T)` は「受け取る値の型は `T`」、戻り値 `: T[]` は「`T` の配列を返す」という意味です。**同じ `T` を使い回す**のがポイントで、「受け取った値と同じ型の配列を返す」と表現できています。
- 中身は `return [x]` で、受け取った値 `x` を1個だけ入れた配列を返すだけです。

> **用語:** ジェネリック関数（generic function） = `<T>` のように「あとから決まる型」を受け取り、いろいろな型で使い回せる関数。

解説2: 呼び出した値に合わせて `T` が自動で決まる

コピー

```
// "本A" は文字列 → T は string と決まり、戻り値は string[]
const a = wrap("本A");
// 123 は数値   → T は number と決まり、戻り値は number[]
const b = wrap(123);
```

- `wrap("本A")` のように**文字列**を渡すと、TypeScriptが「今回の `T` は `string` だ」と自動で判断します。だから戻り値 `a` の型は `string[]`（文字列の配列）になります。
- `wrap(123)` のように**数値**を渡せば、`T` は `number` と決まり、戻り値 `b` は `number[]` になります。
- このように、**1つの関数で「渡した値の型に合わせて」正しい型を保てる**のがジェネリック関数の利点です。`string` 用・`number` 用と別々に関数を作らずに済みます。
- 第9節で学んだ `useState<Book[]>` なども、内部はこのようなジェネリックの仕組みで作られています。「使う側」になるだけなら、`<...>` に中身の型を渡すだけでOKでしたね。

> **用語:** `T` は呼び出し時の値に合わせて自動で決まる（型推論）。明示したいときは `wrap<string>("本A")` と書くこともできる。

---

10. この章のまとめ

この章では、後の章のコードを読むために必要なTypeScriptの基礎を学びました。

- **変数**: `const`（変更しない）/ `let`（変更する）。迷ったら `const`
- **基本の型**: `string`（文字列）/ `number`（数値）/ `boolean`（真偽値）
- **配列** `[ ]` と **オブジェクト** `{ }` でデータをまとめる
- `type` で「本の形（Book型）」のような型に名前を付ける。`?` で省略可能
- **関数** と **アロー関数** `=>`。React Nativeではアロー関数が主流
- `map`（変換）/ `filter`（抽出）は一覧表示で多用する
- `if` と **三項演算子** `条件 ? A : B` で条件分岐
- **ジェネリクス** `<...>`（例: `useState<Book[]>`）は「入れ物の型に"中身の型"を渡す」しくみ
- `async` / `await` でサーバー通信などの「待つ処理」を書く。戻り値は `Promise<...>`

> **全部覚えなくて大丈夫:** ここで紹介した文法は、後の章で実際のコードに何度も出てきます。そのたびにこの第2章に戻って確認すれば、自然と身につきます。次の第3章では、これらを使って画面の部品を作る **React** を学びます。